Examples of Reasoning Quality Patterns

Conversation Log | June 27, 2026

USER

create examples of the prompting patterns in Reasoning quality category

CLAUDE

Here's one example for each of the four patterns under Reasoning Quality — Chain-of-Thought, Self-Critique, Chain-of-Verification, and Devil's Advocate.

Chain-of-Thought

Ask the model to reason step-by-step before giving a final answer, rather than jumping straight to a conclusion — useful whenever the answer depends on working through several dependent considerations.

> *"TextFinder1 currently buffers entire files into memory before scanning for matches. Should I switch to streaming line-by-line reads instead? Think through the tradeoffs step by step before giving a recommendation."*

The model walks through: memory usage scales with file size under buffering, which matters if TextFinder1 ever scans large log files; streaming avoids that but makes multi-line pattern matching across line boundaries harder; most of your current use cases (source files) are small enough that buffering's memory cost is negligible; the real question is whether you anticipate scanning large files in the future. Only after laying that out does it recommend keeping buffering for now, with streaming as a documented option if the use case changes.

Why it matters: the reasoning is visible, so if the model weighted something wrong — say, it didn't know you might add log-file support later — you can correct that input rather than just rejecting an unexplained answer.

Self-Critique / Critic Pattern

Have the model produce an answer, then separately turn around and critique that same answer against explicit criteria, then revise based on its own critique.

> *"Draft the constitution.md naming conventions section for TextFinder2. Then, separately, critique your own draft: are any conventions ambiguous, contradictory, or missing a case a developer would actually hit? Revise based on what you find."*

First pass might produce reasonable-sounding rules — PascalCase for types, snake_case for functions. The critique pass then catches what the first pass missed: it never specified module-file naming, didn't address how generic type parameters should be cased, and one example used snake_case for a constant when the stated rule was SCREAMING_SNAKE_CASE. The revision fixes those gaps.

Why it's distinct from Chain-of-Thought: Chain-of-Thought reasons toward an answer; Self-Critique treats the finished answer as a separate object to be judged, using a second pass with a different posture

(skeptical reviewer instead of drafter).

Chain-of-Verification

After producing an answer, have the model generate specific verification questions about its own claims, then check each one — catching errors that survive a single generation pass because nothing forced a second look at any individual claim.

> *"You just told me Rust's regex crate compiles patterns lazily by default. Before I rely on that, generate 2-3 verification questions about that claim and answer each one honestly — don't just restate the original claim."*

The model might ask itself: "Does Regex::new() actually defer compilation, or compile immediately?" — and on checking, recognize that Regex::new() compiles eagerly; laziness would require something like once_cell/Lazy wrapping it. The original claim gets corrected rather than left standing.

Why it's distinct from Self-Critique: Self-Critique judges the answer holistically ("is this good?"); Chain-of-Verification breaks it into discrete, checkable factual claims and interrogates each one — better suited to catching specific wrong facts rather than structural or stylistic problems.

Devil's Advocate / Red-Teaming

Explicitly prompt for the strongest case against the current direction before finalizing — forcing a real counterargument rather than a token one.

> *"I've decided to use OCaml for TextFinder2 over Haskell and Clojure. Before I commit, argue the strongest case against that choice — not a weak strawman, the actual best argument someone who disagreed would make."*

The model might respond: the strongest case against OCaml here is that its ecosystem for CLI tooling and cross-platform file APIs is thinner than either alternative's, meaning more of your time goes into writing glue code rather than the text-finding logic you actually wanted to compare across languages — which undercuts the project's purpose as a clean idiom comparison if half the OCaml code ends up being workarounds.

Why it's distinct from the others: Chain-of-Verification checks factual claims; Self-Critique reviews the artifact you produced; Devil's Advocate argues against the decision itself, deliberately adopting an opposing stance rather than a neutral reviewing one — useful specifically for catching motivated reasoning or a choice you talked yourself into too quickly.

The four build on each other: Chain-of-Thought makes the reasoning visible, Self-Critique and Chain-of-Verification check that reasoning's output for quality and facts respectively, and Devil's Advocate stress-tests the conclusion by deliberately trying to break it.